

Universidad Nacional Autónoma de México



Facultad de Ingeniería

División de Ingeniería Eléctrica



Laboratorio Computación Gráfica e Interacción

Humano-Computadora (6590)

Grupo (teoría): 02

Grupo (laboratorio): 03

Profesor(teoría): DR. SERGIO TEODORO VITE

Profesor(laboratorio): ING. JOSE ROQUE ROMAN

GUADARRAMA

Semestre 2026-1

Práctica 01 – Introducción a OpenGL

Alumno:

Nombre

Calderón Gutiérrez Victor Emiliano - 423097331

Fecha de entrega: 02/05/2025

Calificación: _____

Práctica 01 – Introducción a OpenGL

1. Introducción y objetivos

Durante la realización de esta práctica se comenzará con el aprendizaje de OpenGL, la cuál es la librería que se utilizará durante el curso y con se lograrán hacer gráficos por computadora. Primero se entender que es OpenGL y algunos de sus comandos básicos.

En el desarrollo de esta práctica se buscará generar un programa el cuál logre dibujar las iniciales del alumno, así como también un fondo el cual se deberá buscar una lógica de programación que permita que este cambia de forma aleatoria cada cierto rango de tiempo. Esta, aunque parezca una práctica sencilla será capaz de retornos para lograr aprender lo básico de esta librería.

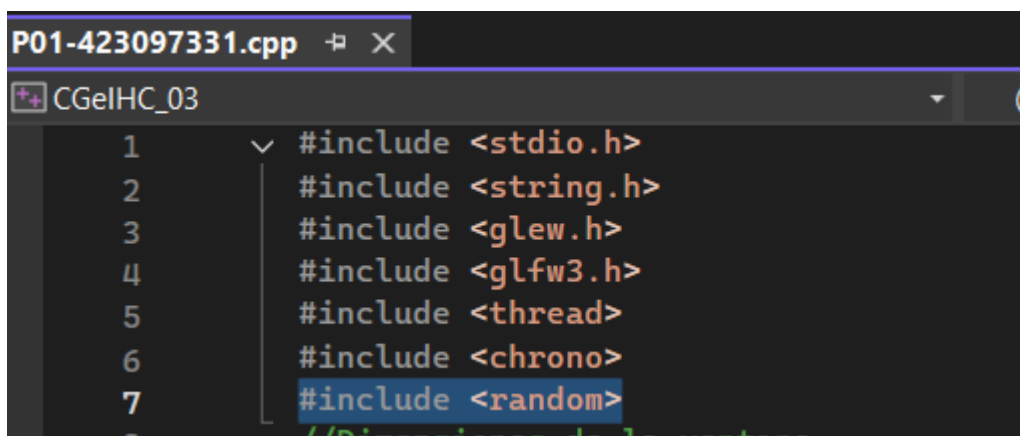
2. Contenido

Como se mencionaba, el alumno debía generar una serie de instrucciones para lograr realizar esta práctica:

1. Ventana cambia el color de fondo de forma random tomando rango de colores RGB y con una periodicidad de 2 segundos.
2. 3 letras iniciales de sus nombres creadas a partir de triángulos, todas las letras son del mismo color.

El código base fue el que se usó el día de la práctica, el código nombrado como *segundo_main.cpp*, el cuál permitía únicamente la impresión de un triángulo con un fondo estático. Se comenzó con la modificación de este código con un par de cosas que nos permitieron completar las tareas. Vayamos por partes.

Para la primera parte, una ventana que cambia de color de forma random. Para esto primero se tuvo que hacer una breve investigación de alguna función *random* y que no dependiera directamente de los ciclos de reloj, se encontró la biblioteca "*random*" la cual contiene una serie de funciones que permite generar valores numéricos aleatorios.



```
P01-423097331.cpp
CGeIHC_03
1  #include <stdio.h>
2  #include <string.h>
3  #include <glew.h>
4  #include <glfw3.h>
5  #include <thread>
6  #include <chrono>
7  #include <random>
8  //Dimensiones de la ventana
```

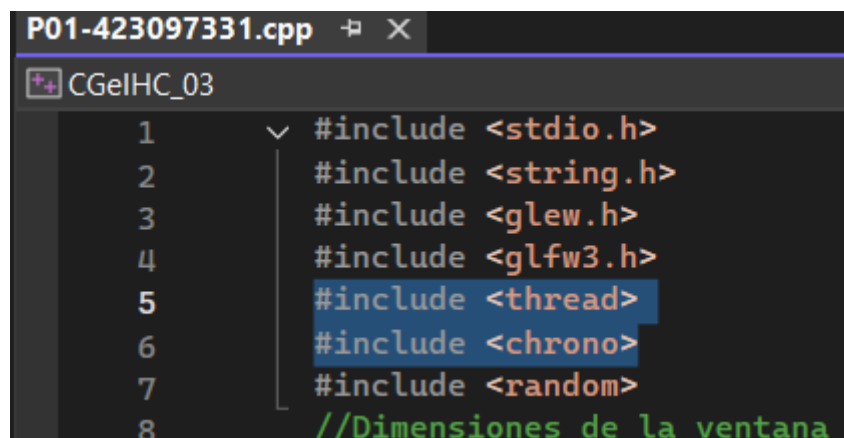
La lógica implementada fue la siguiente:

```
261
262 //Generadores de números aleatorios
263 std::random_device rd;
264 std::mt19937 gen(rd());
265 std::uniform_real_distribution<double> dist(0.0, 1.0);
266
267 //Variables de color
268 float red = dist(gen);
269 float green = dist(gen);
270 float blue = dist(gen);
271
```

Se encontró que *random_device* genera un valor aleatorio el cuál no depende de ningún ciclo de reloj del sistema, posteriormente se usa *mt19937* que funciona usando el algoritmo de *Mersenne Twister* lo cual ayuda a obtener un ciclo pseudoaleatorio. Una vez que se genera así se inicializa con *uniform_real_distribution* para obtener un valor entre 0.0 y 1.0 cada que se llame.

Posteriormente se hace un llamado a esa función creada con las variables para RGB. Así se obtiene el primer número aleatorio.

Ahora, para poder llegar a que se detenga por 2 segundos y que se cree un nuevo color en cada iteración se hizo lo siguiente: primero se hizo uso de las bibliotecas *chrono* y *thread*.



The screenshot shows a code editor window titled "P01-423097331.cpp". The code is in C++ and shows the first eight lines of a program. Lines 1 through 7 are preprocessor directives for including standard libraries: `<stdio.h>`, `<string.h>`, `<glew.h>`, `<glfw3.h>`, `<thread>`, `<chrono>`, and `<random>`. Line 8 is a comment: `//Dimensiones de la ventana`. The code is color-coded, with preprocessor directives in orange and comments in green. The `<thread>` and `<chrono>` lines are highlighted with a blue background.

```
P01-423097331.cpp
CGelHC_03
1  #include <stdio.h>
2  #include <string.h>
3  #include <glew.h>
4  #include <glfw3.h>
5  #include <thread>
6  #include <chrono>
7  #include <random>
8  //Dimensiones de la ventana
```

Luego se usa la siguiente lógica:

```
//Limpiar la ventana
glClearColor(red, green, blue, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

std::this_thread::sleep_for(std::chrono::seconds(2));

//Variables para modificar color de forma aleatoria
red = dist(gen);
green = dist(gen);
blue = dist(gen);
```

Aquí se hace lo siguiente, primero pinta la pantalla con los colores generados aleatoriamente en el paso anterior. Luego se hace uso de *chrono* para pausar la aplicación por exactamente 2 segundos. Y por último vuelve a generar otros datos aleatorios para RGB y al hacer una nueva combinación es como entramos a un entorno aleatorio para generar fondos de pantalla.

Ahora, continuando con la segunda parte, que es generando 3 iniciales de mi nombre. Para esto primero se tuvo que modificar los vectores para todos los triángulos. Para las iniciales que tomé y para la forma en que lo hice decidí tomar 24 triángulos los cuales me ayudaron a generar mis iniciales.

```
GLfloat vertices[] = { ... };
glGenVertexArrays(1, &VAO); //gene
```

Al ser tantas las coordenadas no se agregan en la captura, sin embargo, aquí fue donde se modificó y se puede ver con más detalle en el archivo adjunto a la entrega.

Por último, se tuvo que modificar la cantidad de trazos a dibujar, por lo que se tuvo que simplemente multiplicar la cantidad de triángulos (24) por los lados que tienen (3) y así de llegó al valor de 72 trazos por dibujar.

```
291 glBindVertexArray(VAO);
292 glDrawArrays(GL_TRIANGLES, 0, 72);
293 glBindVertexArray(0);
```

Todo esto en conjunto genera el resultado pedido para esta práctica. Se adjunta video en la entrega con el funcionamiento de este.

3. Comentario/Conclusión

Durante la realización de esta práctica tuve que trabajar con la investigación de como funcionaban algunas funciones para poder tener un mejor entendimiento de lo que ya tenía el programa inicial y posteriormente saber que se le podría agregar.

La complejidad estuvo retadora, ya que esta requirió algo más avanzado a todo lo que se ha visto en la carrera previamente, por lo que significa que fue una práctica bastante avanzada y adecuada al semestre en el que nos encontramos.

Me ayudó bastante para poder entender los conceptos básicos y de paso la investigación y exploración de otras bibliotecas que podrían tener otros usos no solo para esta asignatura.

4. Bibliografía

- *Concurrency support library (since C++11)* - *cppreference.com*. (s. f.).
<https://en.cppreference.com/w/cpp/thread.html>
- *Date and time library* - *cppreference.com*. (s. f.).
<https://en.cppreference.com/w/cpp/chrono.html>
- *std::random_device::random_device* - *cppreference.com*. (s. f.).
https://en.cppreference.com/w/cpp/numeric/random/random_device/random_device